SE EXPERIMENT 1

Aim: Write a detailed Problem Statement for any case study. Justify which process model would be best suited to apply on it.

Title: Smart Parking Management System (SPMS)

Background: As urban populations grow, the number of vehicles increases exponentially, causing extreme congestion in commercial, residential, and corporate multi-storey parking facilities. Traditional parking management relies heavily on manual ticketing, security guard oversight, and a "first-come, first-served" cruising model.

The Problem:

1. **Inefficient Space Utilization & Cruising Time:** Drivers waste an average of 10 to 20 minutes circling multiple floors looking for vacant spots. This leads to high fuel consumption, unnecessary carbon emissions, and driver frustration.
2. **Manual Bottlenecks:** Parking staff struggle to keep track of real-time spot occupancy across multiple levels, resulting in underutilized floors while the ground floor experiences heavy bottlenecks.
3. **Lack of Data Visibility:** Facility managers lack real-time analytics regarding peak hours, average occupancy durations, or revenue leakages.
4. **Hardware-Software Disconnect:** Many existing systems treat sensor arrays (hardware) and booking software as isolated silos rather than an integrated ecosystem.

The Solution: The SPMS aims to bridge this gap by deploying an automated, online software platform integrated with IoT-based proximity/ultrasonic sensors at every single parking unit. The software allows parking staff to immediately allocate an optimal, system-verified vacant spot to incoming customers upon entry, completely eliminating the need for drivers to look around blindly.

Core Modules & System Architecture

To make the theoretical design robust, the system architecture is divided into three concrete layers:

- **The Driver Portal (Frontend Concept):** A user-facing application interface where drivers can register their vehicles, check real-time parking lot occupancy maps before arrival, view allocated slots, and view historical billing data.
- **The Operator/Admin Dashboard:** A high-privilege console used by parking lot management to monitor live parking configurations, override system locks, analyze peak occupancy hours, and audit generated revenue.

- **The Core Logic Engine & Database (Backend Layout):** The central brain that maintains the system state. It handles incoming requests, updates database records when a slot transitions from Available to Occupied, and calculates precise financial tariffs based on check-in and check-out timestamps.

System Flow & Business Logic

1. **Check-In Phase:** A vehicle arrives at the gate. The system captures the vehicle number, checks the database for available slots, updates the state of the selected slot from Available to Occupied, and prints/generates a digital ticket containing a unique ID and an Entry Timestamp.
2. **In-Lot Phase:** The system continuously monitors the database. The slot remains locked under the active vehicle's profile.
3. **Check-Out Phase:** Upon exiting, the ticket ID is scanned. The system fetches the Entry Timestamp, logs the current Exit Timestamp, and passes both values into a billing function. Once the transaction is cleared, the system resets the slot state back to Available.

The Selected Process Model: The Incremental Model

The **Incremental Process Model** breaks the software product down into multiple, linear development cycles called **increments**. Each increment delivers a fully functioning portion of the software, building sequentially upon the previous release until the entire system is complete.

For this project, the theoretical development is mapped into **three distinct increments**:

Increment 1: Core Slot Allocation Engine (The Foundation)

- **Focus:** Core database schemas and structural data flows.
- **Deliverables:** Designing the Users, Slots, and Tickets tables. Creating the algorithmic backend function allocateSlot() and the state machine logic that transitions a parking slot between states (Available, Occupied, Under Maintenance).
- **Testing Focus:** Basic structural verification to ensure database transactions don't fail under concurrent arrival requests.

Increment 2: User & Administrative Interfaces (The Interaction Layer)

- **Focus:** User authentication, dashboards, and access control.
- **Deliverables:** Building the driver interface (UI wireframes for viewing empty spots) and the admin console (UI for monitoring total lot statistics).
- **Integration:** Attaches these interfaces directly onto the core engine built in Increment 1, allowing users to physically trigger the database state changes.

Increment 3: Automated Billing & Analytical Reporting (The Premium Layer)

- **Focus:** Financial transactions, duration math, and reports.
- **Deliverables:** Writing the calculateTariff(entryTime, exitTime) function, handling digital payment success/failure state paths, and generating text-based receipts.
- **Final Assembly:** Completes the product, leaving you with a fully realized, multi-layered software architecture.

Why the Incremental Model is the Most Suitable

Choosing the Incremental Model over other frameworks like Waterfall, Prototype, or Agile Scrum provides critical advantages for a theoretical parking system design:

- **Clean Structural Division for Academic Reports:** A parking system naturally splits into isolated, modular operational layers (Allocation ➔ Interfaces ➔ Billing). The Incremental Model mirrors this perfectly, allowing you to write your SRS and design your Level 1 and Level 2 Data Flow Diagrams (DFDs) page-by-page, module-by-module, keeping your lab documentation extremely clean.
- **Simplified Estimation and Testing Metrics:** In later practicals, trying to evaluate a massive, fully integrated system all at once is overwhelming. With the Incremental Model, you can easily isolate a single function from Increment 1 (like slot state toggling) or Increment 3 (the tariff equation) to draw control flow graphs and write exact test cases without confusing your structural paths.
- **Ideal for Team Task Delegation:** If you are collaborating with peers, this model eliminates conflicting design overlaps. One person can own the theoretical architectural design for the foundational database layout (Increment 1), while another designs the visual wireframes and actor use cases (Increment 2) concurrently.
- **Mitigates Design Over-Complexity:** Unlike evolutionary or prototype models which can loop endlessly, the linear progress of each increment ensures that you stay on schedule to hit every deadline listed in your college lab manual.

Software Requirements Specification

for

Smart Parking Managment System

Version 1.0 approved

Prepared by

Yajat Padmavat

Aditya Patil

Aryan Punalekar

TSEC

<25/07/26>

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this software is to provide drivers, vehicle owners, and parking facility managers with a centralized platform for real-time tracking, reservation, and management of parking spaces . Traditionally, drivers spend considerable time cruising around congested urban areas searching for open parking spots, leading to traffic bottlenecks, fuel wastage, and increased emissions. To eliminate these inefficiencies, this software provides prior visibility into open parking slots and enables cashless pre-booking.

1.2 Document Conventions

- **Headings and Subheadings** are formatted in bold text for hierarchical clarity.
- **Crucial functional conditions** are emphasized in bold.
- Functional priorities follow a descending order from high to low priority across feature definitions.
- Standard software engineering terms and technical acronyms (e.g., API, JWT, MQTT) are referenced throughout and detailed in Appendix A.

1.3 Intended Audience and Reading Suggestions

This document is prepared for:

1. **Software Developers:** To guide the frontend, backend, and hardware-integration development process.
2. **Quality Assurance & Testing Team:** To formulate system test plans, acceptance criteria, and stress tests.
3. **Project Supervisors & Stakeholders:** To verify system functionality against academic and business criteria.

Reading Suggestion: Developers should start with **Section 2 (Overall Description)** followed by **Section 4 (System Features)**, while QA engineers should focus on **Section 3 (External Interfaces)** and **Section 5 (Nonfunctional Requirements)**.

1.4 Product Scope

The **Smart Parking Management System** is a cross-platform solution consisting of a responsive mobile/web application for drivers and a web portal for facility managers. Key features include:

- Pre-arrival real-time slot status inspection.
- Interactive parking spot selection and booking.
- Turn-by-turn navigation guidance to the exact parking spot.
- Automated contactless gate entry using QR codes or IoT sensor verification.
- Payment integration for instant digital tolling/checkout.

Future Enhancements: Automatic License Plate Recognition (ALPR) via AI vision, electric vehicle (EV) charger slot reservation, and predictive AI occupancy modeling

1.5 References

- **React Native / Web Docs:** https://reactnative.dev/
- **Node.js & Express Framework:** https://expressjs.com/
- **PostgreSQL Database Documentation:** https://www.postgresql.org/docs/
- **Google Maps Platform API:** https://developers.google.com/maps

2. Overall Description

2.1 Product Perspective

The Smart Parking Management System acts as a standalone platform connecting drivers directly with commercial and municipal parking infrastructure. By replacing manual paper ticket systems and guesswork with digital sensor streams and web-based slot allocation, the platform cuts entry/exit times, optimizes lot utilization, and eliminates middleman delays.

2.2 Product Functions

- **Live Slot Visibility:** View live counts of free, occupied, and reserved parking spaces prior to driving to the location.
- **Spot Selection & Pre-Booking:** Select precise parking slots (e.g., ground floor, near exit, EV charging slot).
- **Automated Check-in/out:** Mobile QR code generation for scanning at hardware barriers.
- **Integrated Payments:** Calculation of parking fees based on hourly rates with digital receipt issuance.
- **Administrative Operations:** Facility layout configuration, real-time analytics, dynamic pricing adjustments, and manual overrides.

2.3 User Classes and Characteristics

1. **Drivers / Vehicle Owners (Primary Users):** Access live counts, reserve slots, navigate, pay, and track booking history.
2. **Parking Attendants / Facility Managers:** Oversee physical lot spaces, handle manual entry/exit exceptions, and inspect physical sensors.
3. **System Administrators:** Manage user accounts, approve new parking facility additions, set platform service fees, and view financial reports.

2.4 Operating Environment

- **Client Devices:** Mobile devices (iOS 13+, Android 8.0+) and desktop browsers.
- **Web Browsers:** Google Chrome 90+, Mozilla Firefox 85+, Apple Safari 14+, Microsoft Edge.
- **Backend Platform:** Node.js runtime environment hosted on cloud infrastructure (e.g., AWS / Linux Ubuntu 22.04 LTS Server).
- **Database & Memory Cache:** PostgreSQL 15+ for relational records; Redis 7+ for real-time slot state management.
- **Network Minimum:** 3G / 4G / 5G / Wi-Fi connection with at least 1 Mbps throughput.

2.5 Design and Implementation Constraints

- **CO-1:** Project development schedule is capped at 3 months.
- **CO-2:** Frontend must be built using React / React Native for seamless cross-platform performance.
- **CO-3:** Backend REST APIs and WebSockets must be built on Node.js / Express.
- **CO-4:** System interfaces are provided in English.
- **CO-5:** Hardware interaction with gate barriers requires MQTT standard protocol compliance.

2.6 User Documentation

The system will supply user onboarding flows within the mobile application. Context-sensitive help dialogues will assist during reservation, payment, and QR generation. Administrators will receive an inline operational manual covering lot setup and pricing rule adjustments.

2.7 Assumptions and Dependencies

- **AS-1:** Users possess a GPS-enabled smartphone with an active internet connection.
- **AS-2:** Parking lots are equipped with internet-connected entry/exit scanners or ultrasonic/optical slot sensors.
- **DE-1:** Map rendering depends on Google Maps JavaScript and Mobile SDK APIs.
- **DE-2:** Payment handling depends on payment gateway integration (Razorpay / Stripe).

3. External Interface Requirements

3.1 User Interfaces

- **UI-1 (Landing & Map View):** Interactive visual map displaying color-coded pins for parking locations (Green = High availability, Yellow = Limited, Red = Full).
- **UI-2 (Slot Layout Screen):** Visual grid representing individual physical spaces (Available, Reserved, Occupied).
- **UI-3 (Navigation Bar):** Persistent top/bottom navigation bar for quick access to Map, Bookings, Wallet, and Profile.
- **UI-4 (Alerts & Notifications):** Pop-up alerts indicating expiring reservations, successful payments, or pathing warnings.

3.2 Hardware Interfaces

- **HI-1 (Gate Barrier Controller):** Microcontroller units (ESP32/Raspberry Pi) controlling physical barrier arms receiving open/close signals over MQTT.
- **HI-2 (QR Scanners):** Optical scanners at entry/exit points connected via HTTP API endpoints to validate user passes.
- **HI-3**: Ultrasonic/IR proximity sensors installed per parking slot, communicating occupancy data via MQTT or LoRaWAN
- **HI-4**: Outdoor LED display panels showing available slot counts, connected via RS-485 or Ethernet

3.3 Software Interfaces

- **Operating Systems:** Android, iOS, Windows, macOS, Linux.
- **Database Systems:** PostgreSQL database server connected via SSL.
- **External APIs:** Google Maps API (Geocoding & Directions), Payment Gateway APIs.

3.4 Communications Interfaces

- **HTTPS/TLS 1.3:** Secure encryption protocol for all user transactions and REST endpoint interactions.
- **WebSockets (WSS):** Low-latency duplex connection for pushing real-time slot state changes to client screens.
- **MQTT:** Lightweight messaging protocol for sensor data exchange between backend servers and facility gate hardware.

4. System Features

4.1 Authentication and Authorization

4.1.1 Description and Priority

High Priority. Secures user accounts, protects payment information, and enforces role-based permissions (Driver, Manager, Admin).

4.1.2 Response Sequences

1. User enters phone number/email and password.
2. System validates input, verifies hashed credentials against PostgreSQL, and issues a JSON Web Token (JWT).
3. User is directed to the primary map interface.

4.1.3 Functional Requirements

- **REQ-1: Passwords must be hashed using bcrypt prior to storage.**
- **REQ-2: Passwords must be at least 8 characters long, containing 1 numeric character and 1 special character.**

4.2 Real-time Parking Availability Search

4.2.1 Description and Priority

High Priority (Core Feature). Allows drivers to inspect exact open slot counts across parking facilities prior to arrival.

4.2.2 Response Sequences

1. User enters a location or enables current GPS location.
2. System fetches nearby facilities and streams live availability data via WebSockets.
3. User taps a facility to inspect floor plans and open slots.

4.2.3 Functional Requirements

- **REQ-1:** Live slot updates must synchronize with client views within 2 seconds of status change.

4.3 Advance Slot Reservation

4.3.1 Description and Priority

High Priority. Enables users to reserve a chosen slot for a specified timeframe.

4.3.2 Response Sequences

1. User selects a free slot from the floor grid, specifies arrival time and duration, and confirms.
2. System places a temporary 15-minute lock on the slot.
3. Upon payment confirmation, the slot status changes to "Reserved".

4.3.3 Functional Requirements

- **REQ-1:** System must prevent double-booking by locking selected slots atomically in Redis during checkout.

4.4 Navigation & Directions Feature

4.4.1 Description and Priority

Medium Priority. Guides the user from their current GPS location directly to the selected parking facility entrance.

4.4.2 Response Sequences

1. User taps "Navigate" on an active booking.
2. System invokes Google Maps API route overlay within the app.

4.4.3 Functional Requirements

- **REQ-1:** Application must launch external navigation apps (Google Maps, Apple Maps) as a fallback option.

4.5 QR Code Entry & Exit Gate Processing

4.5.1 Description and Priority

High Priority. Facilitates contactless access to parking facilities.

4.5.2 Response Sequences

1. User presents active booking QR code to the scanner at the entry gate.
2. Hardware posts QR payload to backend API for validation.
3. System verifies booking, records check-in timestamp, and triggers gate signal to open.

4.5.3 Functional Requirements

- **REQ-1:** QR codes must use cryptographic timestamps to prevent reuse or static screenshot spoofing.

4.6 Automated Payment Processing

4.6.1 Description and Priority

High Priority. Handles digital payments for reservations and overstay fees.

4.6.2 Response Sequences

1. System calculates total charges based on duration upon exit scanning.
2. Payment modal prompts user to complete transaction via UPI, card, or wallet.
3. Upon success, digital receipt is issued and gate opens.

4.6.3 Functional Requirements

- **REQ-1:** Payment gateway webhooks must update transaction states asynchronously.

4.7 Facility Management & Dynamic Pricing (Admin)

4.7.1 Description and Priority

Medium Priority. Allows managers to adjust pricing rates and block out slots for maintenance.

4.7.2 Response Sequences

1. Manager logs into Admin Portal, selects a facility, and adjusts base hourly rates.
2. Changes save instantly and reflect on driver search views.

4.7.3 Functional Requirements

- **REQ-1:** Rate adjustments cannot affect pre-existing confirmed reservations.

4.8 Booking History & Receipt Generation

4.8.1 Description and Priority

Low Priority. Provides historical logs of past parking sessions with downloadable invoices.

4.8.2 Response Sequences

1. User opens Profile > Booking History.
2. System renders historical list with date, cost, location, and PDF download button.

4.8.3 Functional Requirements

- **REQ-1:** Receipts must be rendered dynamically in PDF format upon request.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **PR-1:** System must support at least 1,000 concurrent WebSocket connections per node without degrading response times beyond 2 seconds.
- **PR-2:** Database queries for slot availability must execute within 100ms under normal traffic conditions.

5.2 Safety Requirements

- **SR-1:** In the event of network or server failure, physical gate barriers must default to manual operation mode to prevent vehicle entrapment.
- **SR-2:** Hardware API tokens and database connection strings must be stored securely using environment variables and secret managers.

5.3 Security Requirements

- **SEC-1:** All database records storing user passwords must be hashed using bcrypt with a salt factor of 10
- **SEC-2:** Transport Layer Security (TLS 1.3) must be enforced across all communications.
- **SEC-3:** Role-based access control (RBAC) must restrict administrative endpoints to authenticated admin tokens only.

5.4 Software Quality Attributes

- **5.4.1 Usability:** Interface layout must adhere to standard mobile design guidelines, requiring at most 3 taps to reach reservation screens.
- **5.4.2 Availability:** System architecture should aim for 99.9% uptime using cloud container auto-scaling.
- **5.4.3 Maintainability:** Modular code structure with thorough API documentation (OpenAPI/Swagger) to simplify future code additions.
- **5.4.4 Testability:** Minimum 80% code coverage across unit and integration testing pipelines.

5.5 Business Rules

- **BR-1:** Reservations hold a 15-minute grace period past the booked start time. If the vehicle fails to enter, the reservation is canceled and a nominal fee is charged.
- **BR-2:** Vehicles overstaying their booked window are billed at an escalated penalty rate per additional hour.

6. Other Requirements

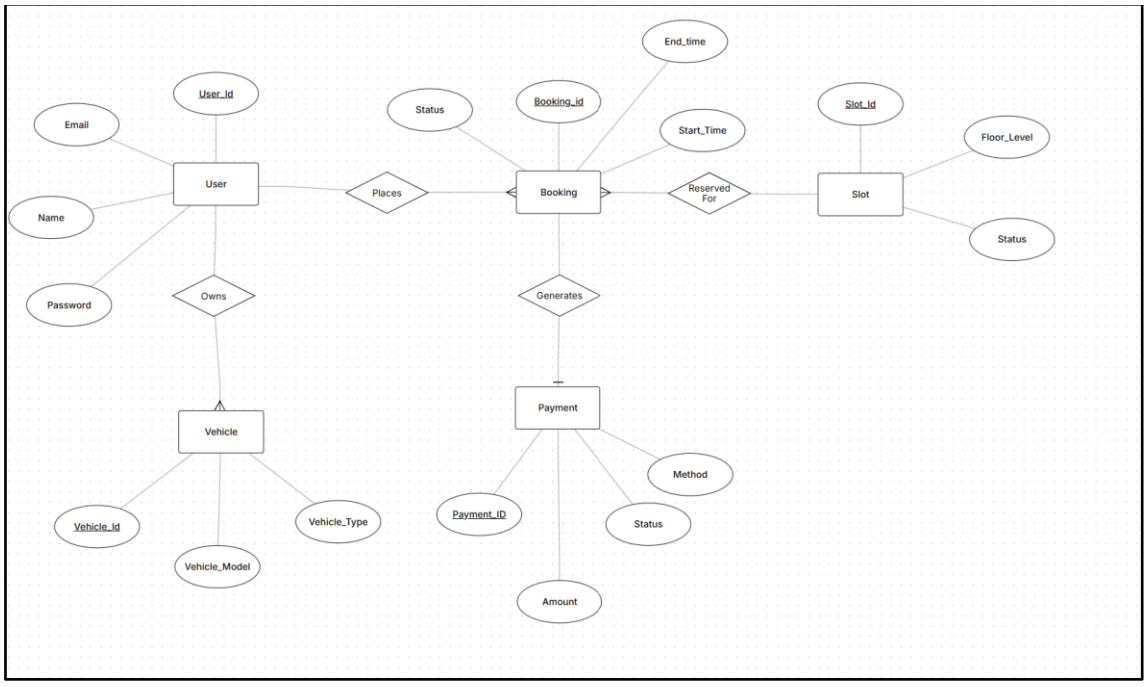
- **Localization:**System timestamps and currency formats must automatically adapt to regional defaults (e.g., IST / INR) .

Appendix A: Glossary

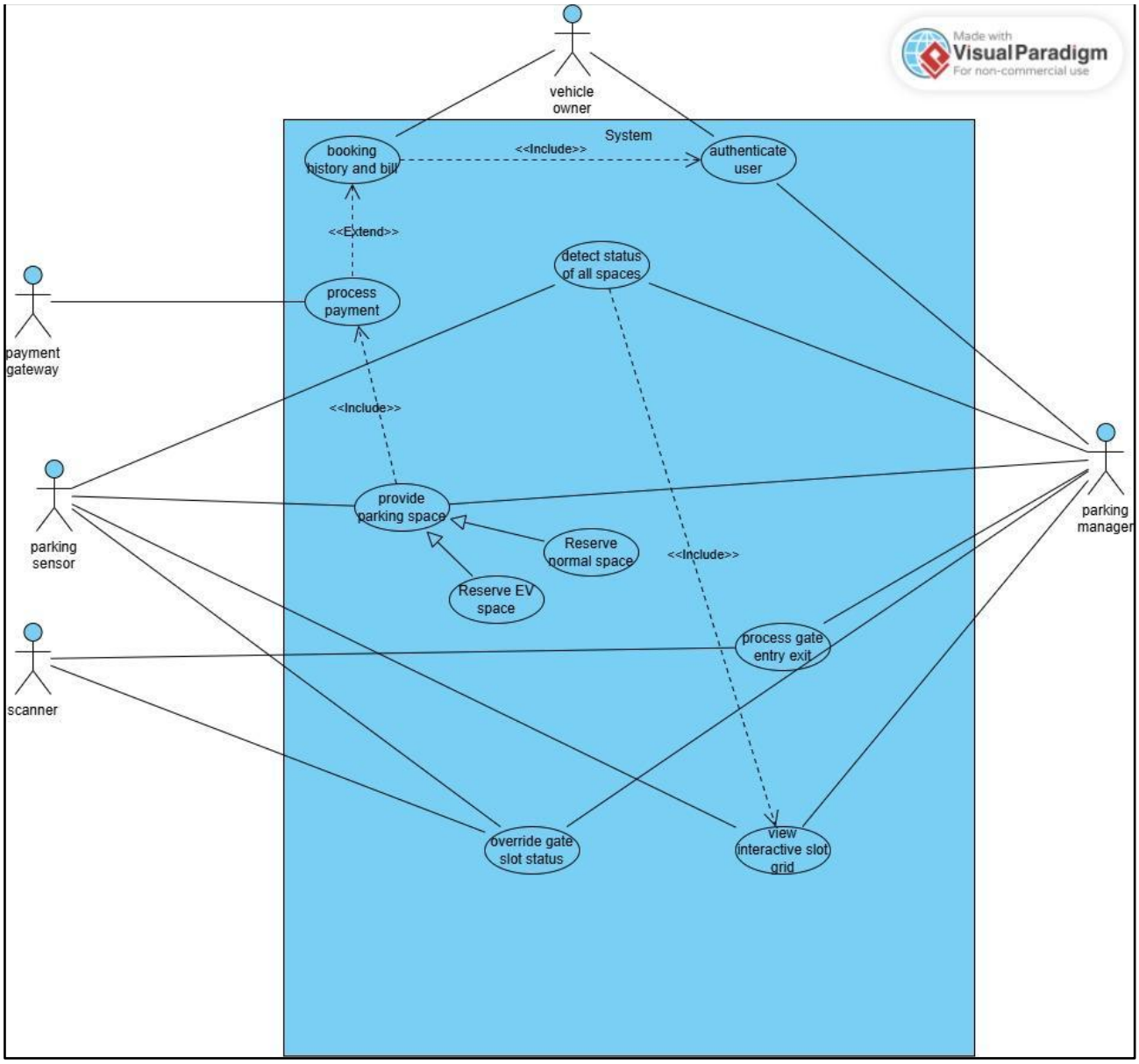
- **API:** Application Programming Interface.
- **GUI:** Graphical User Interface.
- **HTTPS:** Hypertext Transfer Protocol Secure.
- **JWT:** JSON Web Token (Used for secure authentication)
- **MQTT:** Message Queuing Telemetry Transport (Lightweight messaging protocol for IoT sensors).
- **WebSocket:** Computer communications protocol providing full-duplex communication channels over a single TCP connection.

Appendix B: Analysis Models

a) ER diagram



b) Use Case Diagram



Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>

SE EXPERIMENT 3

Aim: Develop Use Case Diagram and Class Diagram for the Smart Parking Management System (SPMS) project using any open-source tool.

Use Case Diagram

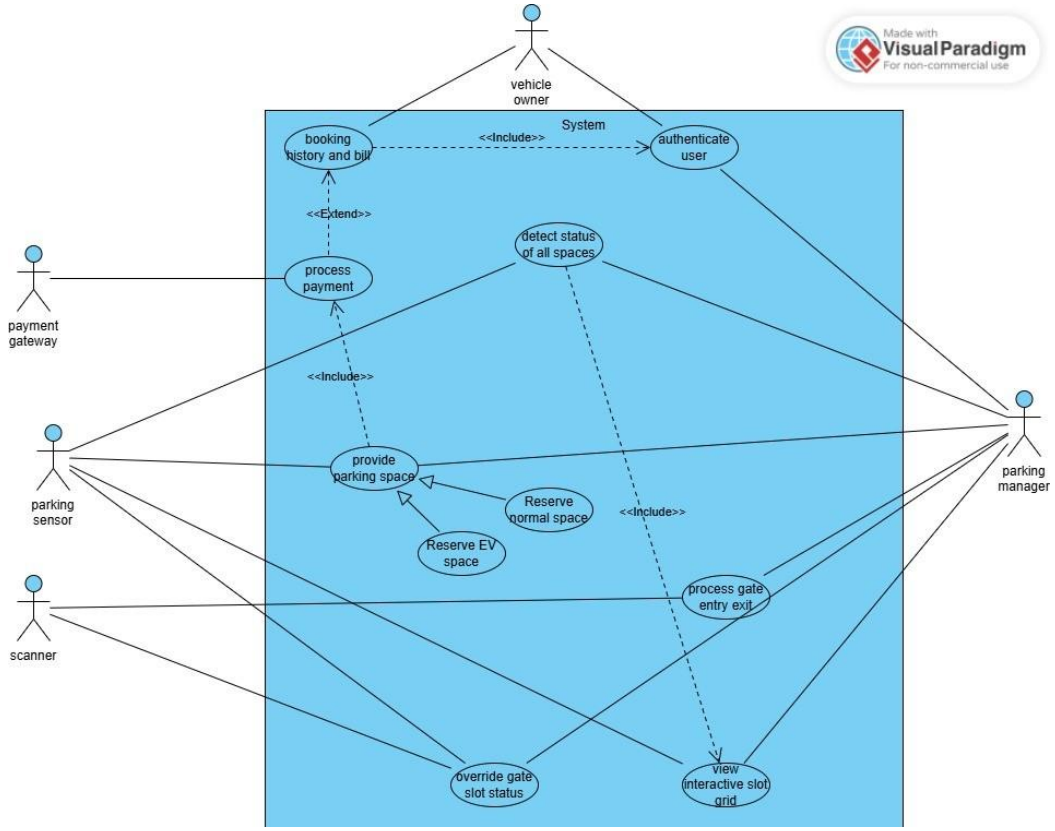
The Use Case Diagram depicts the functional interaction between external actors (human users and hardware/external services) and the system boundary of the Smart Parking Management System.

Actors & Operational Scope:

- Vehicle Owner: Interacts with the system to authenticate credentials, view booking history/bills, and reserve specific parking spaces.
- Parking Manager: Oversees facility operations, views interactive slot grids, detects space statuses, handles entry/exit processing, and manually overrides gate/slot statuses during exceptions.
- Payment Gateway: External service actor responsible for processing payments upon reservation or checkout.
- Parking Sensor: IoT hardware actor supplying status signals to provide real-time parking space data and update space occupancy.
- Scanner: Hardware actor scanning entry/exit passes, connected to gate override and validation flows.

Use Cases & Relationships:

- authenticate user: Core authentication process included (<<Include>>) by booking history and bill.
- booking history and bill: Extended (<<Extend>>) by process payment.
- provide parking space: Generalization parent for Reserve normal space and Reserve EV space. Includes (<<Include>>) process payment.
- detect status of all spaces: Includes (<<Include>>) view (interactive slot) grid for live monitoring.
- process gate entry exit & override gate slot status: Operational management functions connected directly to the Parking Manager, Scanner, and System.



Class Diagram

The Class Diagram models the static structure of the Smart Parking Management System by illustrating system entities, attributes, operational methods, and structural relationships (generalization, association, and dependency).

Class Breakdown & Relationships:

- User (Base Class):
  - *Attributes:* User-id, Name, Email, Password
  - *Method:* Authenticate()
  - *Relationships:* Parent class in a generalization hierarchy with Vehicle Owner and Parking Manager.

- Vehicle Owner (Inherits from User):
  - *Attributes:* Vehicle No.
  - *Methods:* Book_Space(), View History()
  - *Relationships:* Associated with Booking (creates bookings) and Payment.

- Parking Manager (Inherits from User):
  - *Attributes:* Manager-id
  - *Methods:* Detect_Status(), Override_Status()
  - *Relationships:* Associated with Parking Space, Sensor, and Gate for administrative control.

- Parking Space (Base Class):
  - *Attributes:* Space-id, Location, Status
  - *Methods:* Check_Availability(), Update_Status()

- *Relationships:* Parent class generalized into Normal Space and EV Space. Associated with Booking and Sensor.

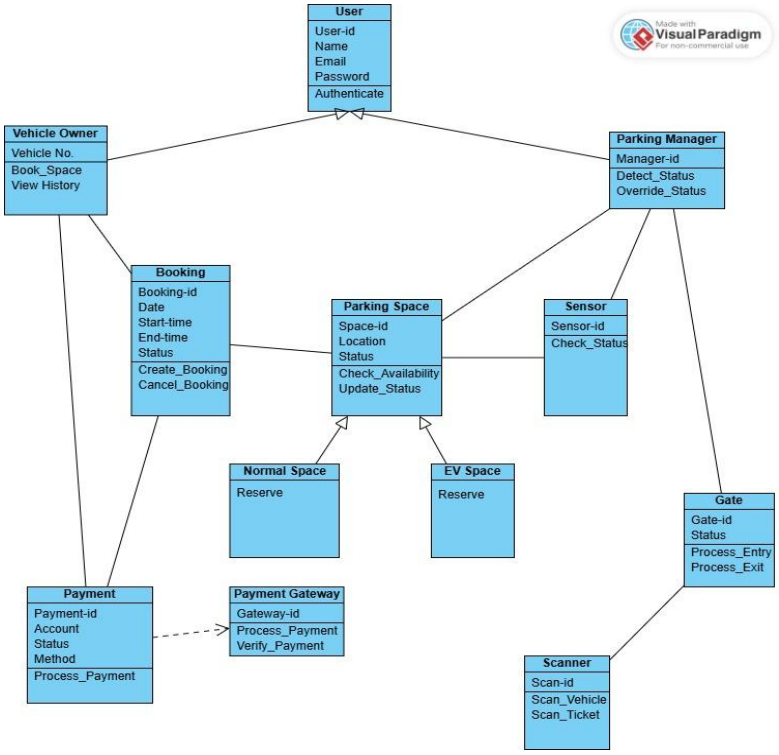
- Normal Space & EV Space (Inherit from Parking Space):
  - *Methods:* Reserve()

- Booking:
  - *Attributes:* Booking-id, Date, Start-time, End-time, Status
  - *Methods:* Create_Booking(), Cancel_Booking()
  - *Relationships:* Links Vehicle Owner with Parking Space and triggers Payment.

- Payment & Payment Gateway:
  - *Payment Attributes:* Payment-id, Account, Status, Method
  - *Payment Method:* Process_Payment()
  - *Payment Gateway Attributes & Methods:* Gateway-id, Process_Payment(), Verify_Payment()
  - *Relationships:* Payment has a dependency relationship (--->) pointing to Payment Gateway.

- Sensor:
  - *Attributes:* Sensor-id
  - *Method:* Check_Status()
  - *Relationships:* Associated with Parking Space and managed by Parking Manager.

- Gate & Scanner:
  - *Gate Attributes & Methods:* Gate-id, Status, Process_Entry(), Process_Exit()
  - *Scanner Attributes & Methods:* Scan-id, Scan_Vehicle(), Scan_Ticket()
  - *Relationships:* Scanner is associated with Gate, which is controlled by Parking Manager.



Result:

The Use Case Diagram and Class Diagram for the Smart Parking Management System (SPMS) were successfully designed and analyzed using UML modeling tools. The diagrams accurately captured system actor interactions, structural class hierarchies, member attributes, methods, and entity relationships required for complete domain modeling.

Experiment 4

Aim: To draw the Data Flow Diagram (DFD) for the Smart Parking Management System using an open-source diagramming tool, representing the flow of data between external entities, system processes, and data stores.

Theory:

Working of the DFD

The **Vehicle Owner** provides credentials to the **User Authentication** process. The system validates the credentials against the user database and generates a JWT session token after successful authentication.

The **Parking Space Status Management** process receives occupancy information from parking sensors and maintains the current status of available, occupied, and reserved spaces. Real-time availability is then provided to the vehicle owner through the application. The SRS specifies that live slot updates should synchronize with client views within two seconds of a status change.

The **Advance Slot Reservation** process receives the user's selected slot, arrival time, and duration. The system temporarily locks the selected slot for 15 minutes and changes its status to **Reserved** after payment confirmation. Redis is used to prevent double-booking during checkout.

The **Navigation Guidance** process uses the active booking and invokes the **Google Maps API** to provide navigation to the selected parking facility.

During entry, the **Gate Entry & Exit Processing** process receives QR scan data from the scanner. The system validates the booking, records the check-in timestamp, and sends a signal to open the gate.

The **Automated Payment Processing** process calculates the parking charges based on the parking duration. The user completes the payment through the payment gateway, after which the transaction status is updated and a digital receipt is generated.

The **Booking History & Receipt Generation** process allows users to view previous parking sessions and generate downloadable PDF receipts.

Finally, the **Facility Management & Dynamic Pricing** process allows the Parking Manager to modify parking rates and block slots for maintenance. Changes are reflected in subsequent parking searches, while confirmed reservations remain unaffected by new pricing rules.

DFD Levels

Level 0 DFD / Context Diagram

At Level 0, the **Smart Parking Management System** is represented as a single process. The external entities interacting with the system are:

- Vehicle Owner
- Parking Manager
- Sensor
- Payment Gateway
- Google Maps API

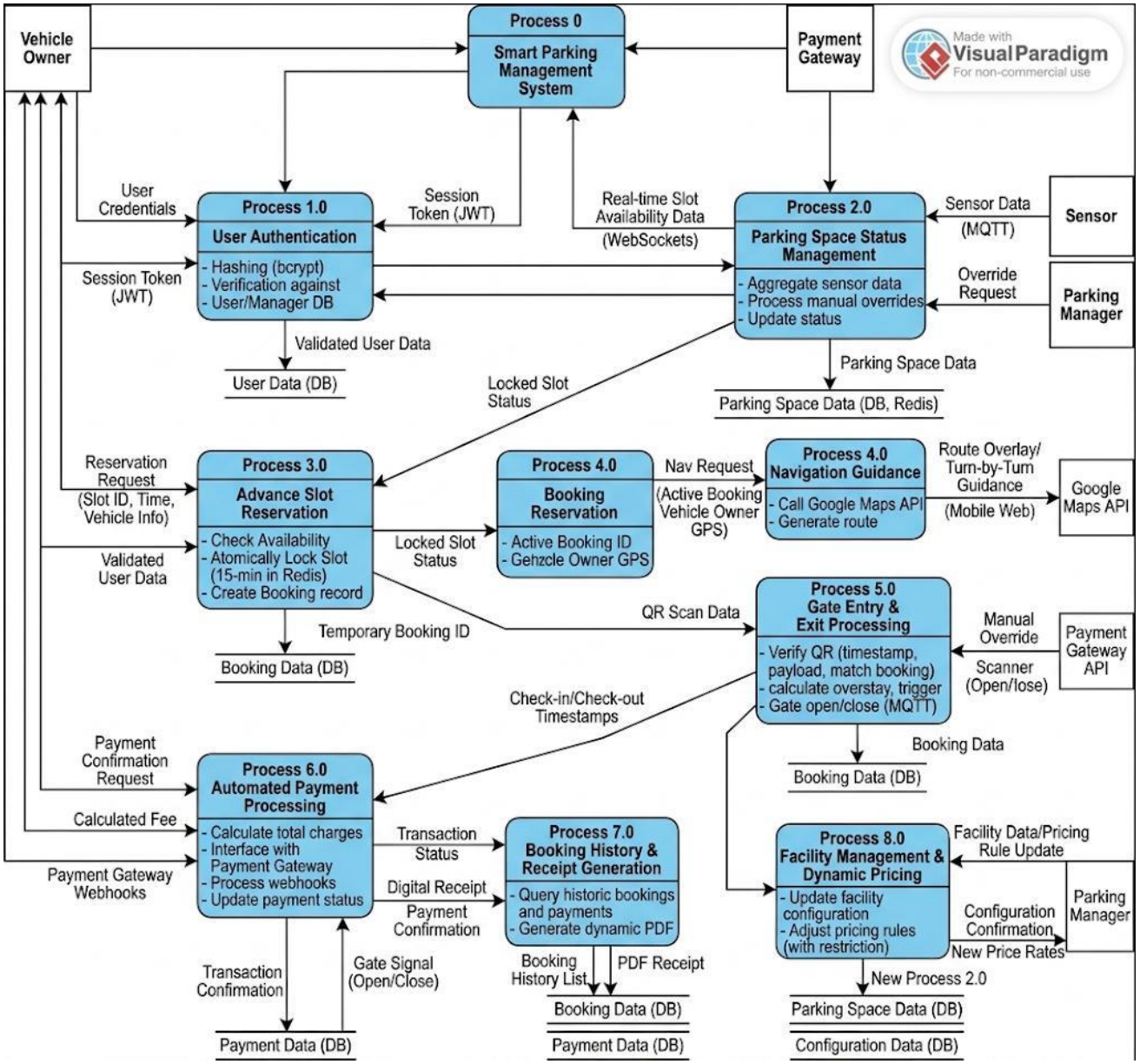
The diagram shows the major input and output data flows between these entities and the complete system.

Level 1 DFD

At Level 1, the main system is decomposed into individual processes such as:

- User Authentication
- Parking Space Status Management
- Advance Slot Reservation
- Booking Reservation
- Navigation Guidance
- Gate Entry & Exit Processing
- Automated Payment Processing
- Booking History & Receipt Generation
- Facility Management & Dynamic Pricing

This provides a more detailed representation of how data moves through the individual functional modules.



Tool Used:

diagrams.net (Draw.io) – an open-source/free diagramming tool used for creating the DFD.

Result:

The Data Flow Diagram for the Smart Parking Management System was successfully designed using an open-source diagramming tool. The DFD represents the flow of data between vehicle owners, parking managers, sensors, external services, system processes, and databases involved in parking reservation, monitoring, navigation, entry/exit, payment, and facility management.

SE EXPERIMENT 5

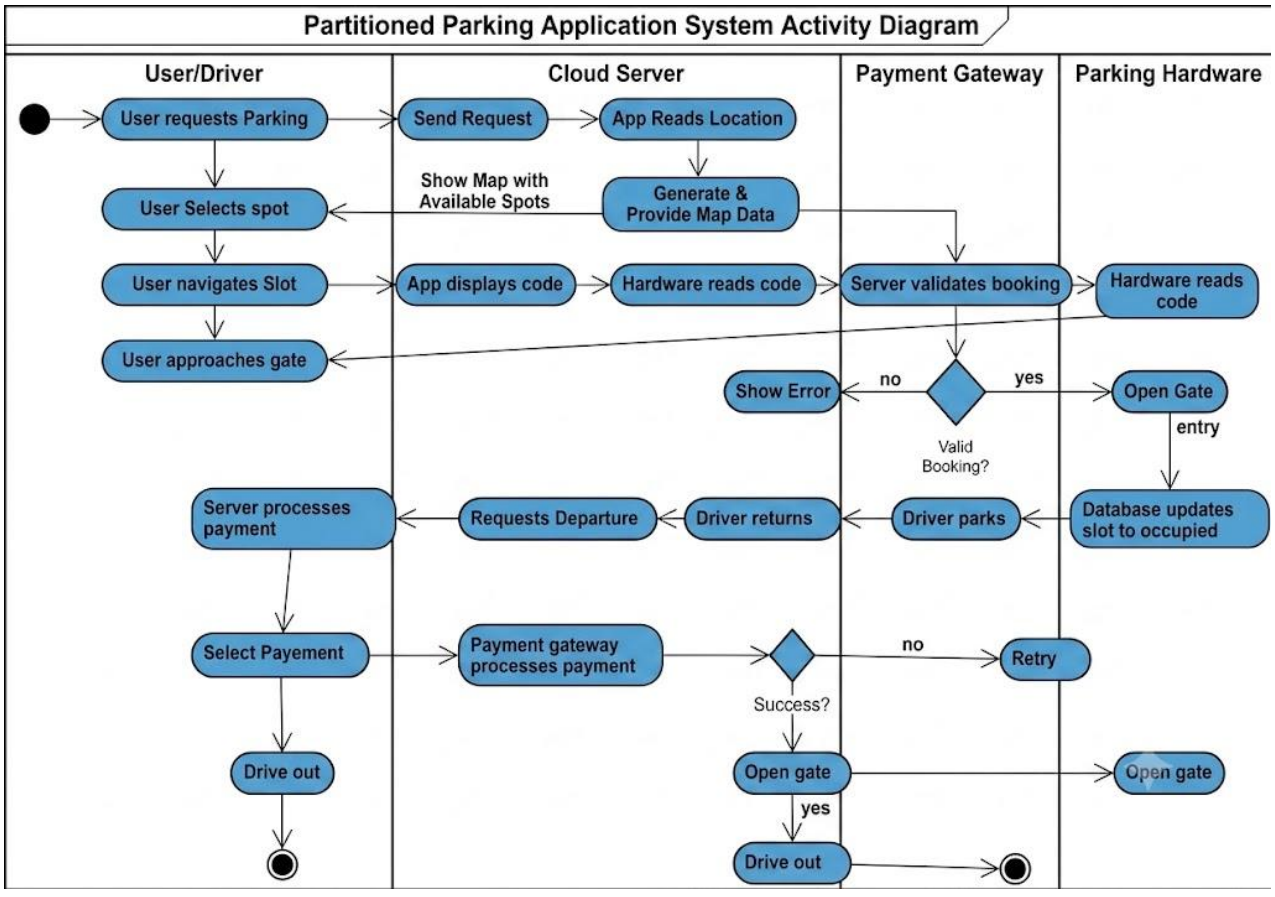
Aim: Develop Activity and State Transition diagram for the Smart Parking Management System using any opensource tool.

1. Activity Diagram

The activity diagram models the end-to-end user workflow across four system partitions (swimlanes): User/Driver, Cloud Server, Payment Gateway, and Parking Hardware.

Swimlane Description & Operational Flow:

- User / Driver: Requests parking, selects a spot, navigates to the gate, parks, and authorizes checkout payment upon departure.
- Cloud Server: Processes location data to stream spot availability, validates QR passes, and dynamically updates slot occupancy.
- Parking Hardware: Scans QR passes at gates, syncs validation requests via MQTT/HTTP, and operates entry/exit barriers.
- Payment Gateway: Asynchronously processes parking fees, manages retries on failure, and issues payment confirmations.



2. State Diagram

The state transition diagram models the complete operational states and event triggers of an active parking session.

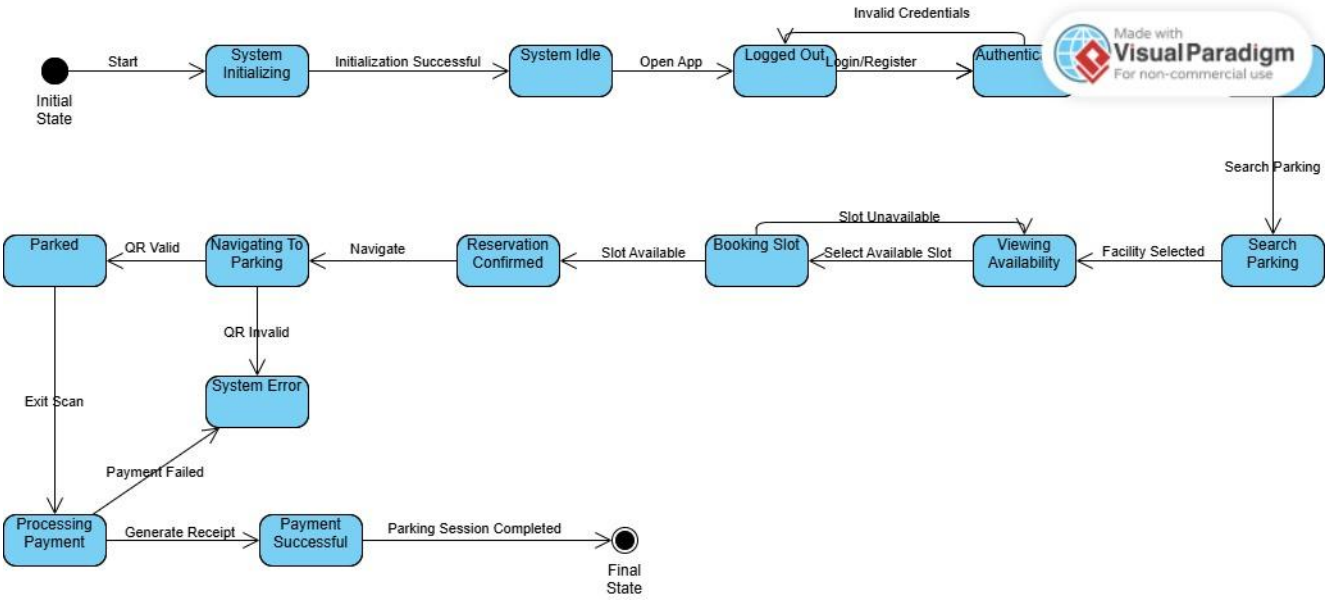
State Breakdown & Transition Logic:

- System Startup & User Login: Boots from Initial State to System Idle, then shifts to Logged Out on app launch until successful login transitions to Authenticated.

- Discovery & Slot Reservation: Advances through Search Parking to Viewing Availability, moving to Booking Slot to lock a spot—looping back if unavailable or proceeding to Reservation Confirmed with a QR pass once booked.

- Navigation, Check-In & Parking: Transitions from Navigating To Parking to Parked upon valid QR scan at the barrier (or routes to System Error if rejected).

- Exit, Billing & Termination: Enters Processing Payment via exit scan, resolving to System Error on failure or Payment Successful on approval, and finally reaches Final State as the vehicle departs.



Result:

The Activity Diagram and State Transition Diagram for the Smart Parking Management System (SPMS) were successfully designed and analyzed using UML modeling tools. The diagrams effectively captured the partitioned operational workflow across actors and accurately mapped the entity state lifecycle from system initialization to checkout completion.

SE EXPERIMENT 6

Aim: To Study software estimation techniques like Function Point Analysis and COCOMO model and calculate the cost for Smart Parking Management System (SPMS).

1. Function Point Analysis (FPA)

Function Point Analysis is a structured, technology-independent metric developed to quantify the software functionality delivered to a user based on external logical requirements. It evaluates functionality across five primary Information Domain Characteristics:

- External Inputs (EI): Elementary user/system data input transactions that update internal logical files.
- External Outputs (EO): Elementary processes that compute and present derived data or reports to users.
- External Inquiries (EQ): Input-output combinations resulting in immediate data retrieval without database modification.
- Internal Logical Files (ILF): User-identifiable logical data groups maintained entirely within system boundaries.
- External Interface Files (EIF): Data groups referenced by the application but maintained by external software systems.

The calculation derives Unadjusted Function Points (UFP) and adjusts them using the Value Adjustment Factor (VAF) based on 14 General System Characteristics (Fi):

$$VAF = 0.65 + 0.01 \times (\Sigma Fi)$$

$$Final\ FP = UFP \times VAF$$

2. COCOMO II (Object Point & Size Estimation Model)

COCOMO II (Constructive Cost Model) Early Prototyping/Application Composition model estimates project effort from direct system components (screens, reports, 3GL programming modules). The raw Object Points (OP) are adjusted for component reuse to obtain New Object Points (NOP):

$$NOP = OP \times [(100 - \%\ reuse) / 100]$$

$$Effort\ (Person\text{-}Months) = NOP / PROD$$

where PROD represents developer team productivity determined by developer experience and tool/environment maturity.

Information Domain Characteristics Table (UFP Calculation)



FP CALCULATOR

Domain Characteristic Table

MEASUREMENT PARAMETER	COUNT (value >= 0)	WEIGHTING FACTOR Simple	Average	Complex
Number of User Input	6	○	◉	○
Number of User Outputs	3	○	◉	○
Number of User Inquiries	3	○	◉	○
Number of Files	4	○	◉	○
Number of External Interfaces	2	○	◉	○

Complexity Adjustment Table | FP Calculation

Unadjusted Function Points (UFP) = 24 + 15 + 12 + 40 + 14 = 105

General System Characteristics (Complexity Adjustment Table)

Complexity Adjustment Table

ITEM	COMPLEXITY ADJUSTMENT QUESTIONS	SCALE No Influence 0	1	2	3	4	Essential 5
1	Does the system require reliable backup and recovery?	○	○	○	◉	○	○
2	Are data communications required?	○	○	○	○	○	◉
3	Are there distributed processing functions?	○	○	○	○	◉	○
4	Is performance critical?	○	○	○	○	◉	○
5	Will the system run in an existing, heavily utilized operational environment?	○	○	○	◉	○	○
6	Does the system require on-line data entry?	○	○	○	○	◉	○
7	Does the on-line data entry require the input transaction to be built over multiple screens or operations?	○	○	○	◉	○	○
8	Are the master files updated on-line?	○	○	○	○	○	◉
9	Are the inputs, outputs, files or inquiries complex?	○	○	○	◉	○	○
10	Is the internal processing complex?	○	○	○	◉	○	○
11	Is the code to be designed reusable?	○	○	○	◉	○	○
12	Are conversion and installation included in the design?	○	○	◉	○	○	○
13	Is the system designed for multiple installations in different organizations?	○	○	○	○	◉	○
14	Is the application designed to facilitate change and ease of use by the user?	○	○	○	○	◉	○

Domain Characteristic Table | FP Calculation

Calculation of Adjusted Function Points:

- Total Degree of Influence (Σ Fi) = 3 + 5 + 4 + 4 + 3 + 4 + 3 + 5 + 3 + 3 + 3 + 2 + 4 + 4 = 50
- Value Adjustment Factor (VAF) = 0.65 + 0.01 × (50) = 0.65 + 0.50 = 1.15
- Project Function Points (FP) = UFP × VAF = 105 × 1.15 = 120.75 FP

FP Calculation

NOTE: For any updates made on any of the entries, always click the 'Calculate Function Points' button to recalculate function points value.

Reset / Clear all form entries Calculate Function Points

RESULT	
PROJECT FUNCTION POINTS	120.74999999999999

Top of Page | Domain Characteristic Table | Complexity Adjustment Table

Cost Estimation via Standard Function Point Model:
• Assumption: 1 FP = ₹1,450 units (standard industry reference model)
• Estimated Total Cost = FP × ₹1,450 = 120.75 × ₹1,450 = ₹1,75,087.50

3. COCOMO Model & Size/Effort Estimation

1. Object Point Count & Complexity Classification

• Screens Breakdown:
1. Authentication & Registration Screen — Simple (Weight: 1)
2. Live Slot Availability & Map View Screen — Medium (Weight: 2)
3. Interactive Parking Grid & Reservation Modal — Medium (Weight: 2)
4. Payment & Checkout Screen — Medium (Weight: 2)
5. Facility Manager Override & Configuration Dashboard — Difficult (Weight: 3)

• Reports Breakdown:
1. Dynamic Parking Bill & PDF Invoice — Simple (Weight: 2)
2. Daily Facility Analytics & Revenue Report — Medium (Weight: 5)

• 3GL Components Breakdown:
1. Node.js / Express Backend & WebSockets Real-Time Engine — Difficult (Weight: 10)
2. MQTT Broker & IoT Sensor Hardware Interface Engine — Difficult (Weight: 10)

Complexity Weights and Object Point Matrix

Complexity Weights and Object Point Matrix

Object Type	Simple	Medium	Difficult	Count	Total OP
Screens	1 (wt 1)	3 (wt 2)	1 (wt 3)	5	10
Reports	1 (wt 2)	1 (wt 5)	0 (wt 8)	2	7
3GL Components	0 (wt —)	0 (wt —)	2 (wt 10)	2	20

Total Object Points (OP) = 10 + 7 + 20 = 37

2. New Object Point (NOP) Calculation:
• Code Reusability (% reuse) = 15% (standard UI boilerplates, Express routers, client SDKs)
• NOP = OP × [(100 - % reuse) / 100] = 37 × [(100 - 15) / 100] = 37 × 0.85 = 31.45

New Object Point = 31.45

3. Developer Productivity Rate (PROD) & Effort Calculation:

- Developer Experience = Nominal / Medium (13 NOP/month)
- Development Environment Maturity = Nominal / Medium (13 NOP/month)

Productivity Rate (PROD) = (13 + 13) / 2

= 13 NOP / Person-Month

- Estimated Effort = NOP / PROD = 31.45 / 13 = 2.42 Person-Months

Efforts for project development is 2.42 person months.

4. Size Estimation (KLOC):

- Using calculated FP = 120.75 and 3GL conversion ratio (1 FP = 45 LOC for Node.js or JavaScript):
- Estimated Codebase Size = 120.75 × 45 = 5,433.75 LOC = 5.434 KLOC

Estimated Codebase Size = 5.434 KLOC

5. Schedule & Labor Cost Estimation:

- Development Time (TDEV) = 2.5 × (Effort)^0.35 = 2.5 × (2.42)^0.35 = 3.40 Months
- Labor Cost Estimation (at standard rate of ₹40,000 / Person-Month):
- Total Labor Cost = 2.42 Person-Months × ₹40,000

= ₹96,800.00

Total Labor Cost = ₹96,800.00

Conclusion:

Function Point Analysis and the COCOMO II Object Point model were successfully studied and applied to the Smart Parking Management System (SPMS). The system was evaluated at 120.75 Function Points, corresponding to an estimated codebase size of 5.434 KLOC. The required development effort was determined to be 2.42 Person-Months across an estimated schedule of 3.40 calendar months. Total project costs were calculated at ₹1,75,087.50 using the standard FP rate and ₹96,800.00 in direct labor costs. These metrics provide reliable baselines for sizing, budgeting, and project scheduling.

SE EXPERIMENT 8

Aim: To prepare a project schedule and project plan for the Smart Parking Management System (SPMS) using a project management tool (Gantt Chart).

Theory:

1. Software Project Scheduling

Project scheduling is an essential software management activity that distributes estimated project effort across a planned project timeline by allocating resources to specific tasks and tracking dependencies. For complex IoT and cloud ecosystems like the Smart Parking Management System (SPMS), scheduling coordinates parallel development between embedded hardware layers, distributed cloud microservices, and client user interfaces while adhering to delivery constraints

2. Work Breakdown Structure (WBS)

A WBS hierarchically decomposes the total project scope into manageable, deliverable-oriented tasks. It organizes SPMS into six phases: Architecture, IoT Edge, Cloud Backend, UI, Testing, and Deployment. Each phase defines clear work packages that prevent scope creep and establish team accountability. It serves as the structural foundation for sequencing task dependencies and scheduling timelines.

3. Gantt Chart Representation

A Gantt chart visually plots scheduled project activities as horizontal bars along a time axis.

The position and length of each bar illustrate start dates, completion dates, and overall task durations. Precedence links depict technical dependencies, showing which tasks require prerequisites to finish. Milestone markers indicate key review gates, system integration points, and the final go-live date.

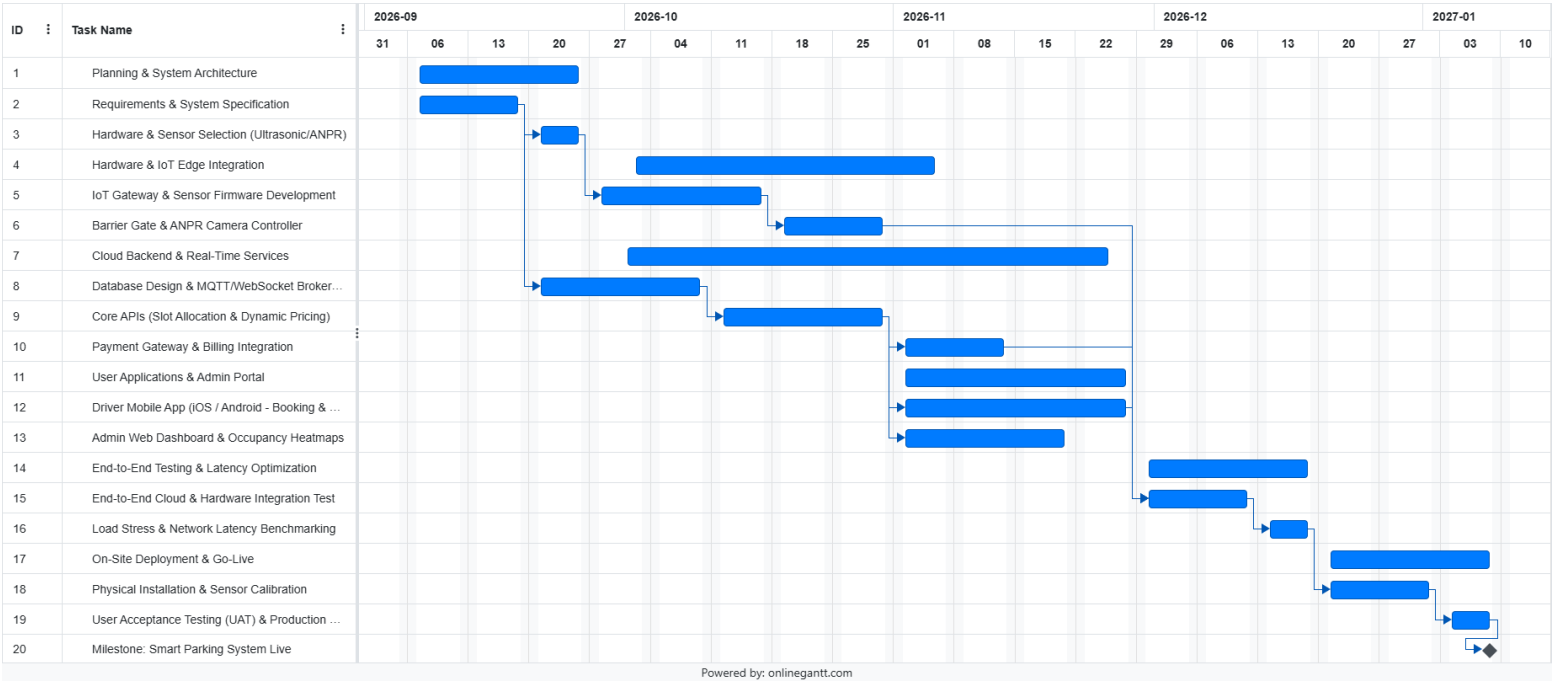
4. Critical Path Method (CPM)

The Critical Path identifies the longest continuous sequence of dependent tasks across the schedule. It dictates the absolute minimum calendar duration needed to complete the overall system deployment. Tasks along this sequence possess zero float, meaning any delay directly pushes back the final release date. It highlights critical milestones, like IoT firmware and integration testing, to prevent project overruns.

Gantt Chart

A Gantt chart is a horizontal bar chart used in project management to visually track tasks and activities against a calendar timeline. Originally developed by Henry Gantt, it maps scheduled project components into discrete execution blocks, allowing engineering teams to evaluate progress, manage workload distribution, and handle task handoffs.

- **Key Components & Visual Elements:**
  - **Time Axis (Horizontal):** Depicts calendar units (days, weeks, or months) spanning the total lifecycle from project kickoff to system sign-off.
  - **Task List (Vertical):** Outlines individual work packages, grouped hierarchically by Work Breakdown Structure (WBS) phases.
  - **Activity Bars:** Horizontal spans where the left edge marks task start date, the right edge marks target completion, and bar length represents estimated duration.
  - **Precedence & Dependencies:** Directed link arrows show constraints between predecessors and successors (e.g., Finish-to-Start), ensuring dependent modules begin only after prerequisite deliverables finish.
  - **Milestones:** Zero-duration markers (often rendered as diamonds) indicating key transition points, major integration gates, or client approvals.
- **Advantages in Software Engineering:**
  - Clarifies parallel development tracks across distinct disciplines (such as embedded IoT firmware and cloud backend APIs) running alongside one another.
  - Highlights concurrency bottlenecks and prevents resource over-allocation across team members.
  - Identifies schedule slippages early, allowing project managers to apply fast-tracking or crashing techniques before deadlines are breached.



Conclusion: The project schedule for the Smart Parking Management System (SPMS) was organized and mapped using an online project management tool. The 20 project tasks were organized across six WBS phases spanning September 2026 to January 2027. Through Gantt chart tracking and Critical Path analysis, hardware-software integration dependencies were defined to keep project execution aligned with the target deployment deadline.

SE EXPERIMENT 8

Aim: To prepare RMMM plan for the Smart Parking Management System (SPMS).

Theory:

Planning the risk management

The proactive strategy for risk estimation is used which helps us in identifying the possible threats that can occur during the project well in advance. Accordingly, steps to avoid, monitor and manage the risk are to be carried out and noted down in the form of RMMM plan.

Example:

THE RMMM PLAN

A risk management strategy can be included in the software project plan or the risk management steps can be organized into a separate Risk Mitigation, Monitoring and Management Plan. The RMMM plan documents all work performed as part of risk analysis and is used by the project manager as part of the overall project plan. Some software teams do not develop a formal RMMM document. Rather, each risk is documented individually using a risk information sheet (RIS). In most cases, the RIS is maintained using a database system, so that creation and information entry, priority ordering, searches, and other analysis may be accomplished easily.

Once RMMM has been documented and the project has begun, risk mitigation and monitoring steps commence. As we have already discussed, risk mitigation is a problem avoidance activity. Risk monitoring is a project tracking activity with three primary objectives:

1. To assess whether predicted risks do, in fact, occur.
2. To ensure that risk aversion steps defined for the risk are being properly applied.
3. To collect information that can be used for future risk analysis.

In many cases, the problems that occur during a project can be traced to more than one risk. Another job of risk monitoring is to attempt to allocate origin (what risk(s) caused which problems throughout the project).

Identifying Risks:

Risk ID	Risk Summary	Probability	Impact	Risk Exposure
1	Gate Barrier Lockup Trapping Vehicles	40%	1	Rs. 64,000/month
2	Concurrent Slot Double-Booking in Redis	65%	2	Rs. 52,000/month
3	QR Code Spoofing & Revenue Leakage	55%	2	Rs. 49,500/month
4	Schedule Slippage Beyond 3-Month Constraint	75%	3	Rs. 41,250/month

Impact:

1 – Catastrophic
2 – Critical
3 – Marginal
4 – Negligible

Risk Exposure:

Risk: Gate Barrier Lockup Trapping Vehicles
The automated gate barrier system relies on microcontroller units (ESP32/Raspberry Pi) and optical scanners receiving open/close signals via MQTT. In the event of a system hang, power fluctuation, or network drop, the barrier arm could remain locked shut. This traps incoming and departing vehicles inside or outside the facility, completely halting multi-storey parking flow and causing severe traffic backlogs on connecting city streets.

Risk Probability: 80%

Risk Impact:
The risk impact is catastrophic (Scale 1) as physical vehicle entrapment directly violates basic facility safety requirements (SR-1), incurs commercial legal liabilities, and halts revenue operations completely.
The risk can be mitigated by installing spring-loaded fail-safe hardware relays that disengage the barrier lock into manual swing mode upon loss of heartbeat signals, alongside dedicated UPS units and emergency manual override switches manned by attendants. An on-site electrical support and triage setup costs Rs. 1,60,000/month.

Risk Exposure: $\text{RE} = 0.40 \times \text{Rs. } 1{,}60{,}000 = \text{Rs. } 64{,}000/\text{month}$

Risk Information Sheet

Risk ID: 1	**Date:** 12/09/2026	**Probability:** 40%	**Impact:** Catastrophic
Description: Microcontroller barriers or MQTT broker links crash, locking physical barrier arms shut and trapping vehicles within the facility.			
Refinement/Context: **Sub condition 1:** MQTT messaging drops or power failures leave microcontrollers unresponsive. **Sub condition 2:** Entry and exit blockages back up traffic onto access roads, creating critical safety hazards.			
Mitigation/Monitoring: 1. Implement fail-safe relays defaulting barriers to manual mode during network/power outage (SR-1). 2. Deploy uninterruptible power supply (UPS) batteries and mechanical manual overrides at every gate. 3. Monitor heartbeat telemetry every 5 seconds.			
Management: RE computed to be Rs. 64,000 per month.			
Current Status: Mitigation steps to be initiated.			
Originator: Aditya Patil		**Assigned:** Aryan Punalekar	

Risk Exposure:

Risk: Concurrent Slot Double-Booking in Redis
The system provides advance slot reservation with an interactive floor grid, placing a temporary 15-minute hold on a slot while checkout proceeds. When multiple drivers target the same high-demand slot simultaneously during peak arrival rushes, a race condition may cause the system to confirm the same physical parking spot for two vehicles.

Risk Probability: 65%

Risk Impact:
The risk impact is critical (Scale 2) as arriving motorists find their reserved space occupied by another car, degrading trust, creating facility bottlenecks, and generating operational disputes.

The risk can be mitigated by utilizing Redis atomic distributed locking (SETNX) with a strict 15-minute Time-To-Live (TTL) alongside PostgreSQL serializable transaction isolation. Dedicated database concurrency optimization and cloud caching services cost Rs. 80,000/month.

Risk Exposure: RE $= 0.65 \times$ Rs. 80,000 $=$ Rs. 52,000/month

Risk Information Sheet

Risk ID: 2	**Date:** 12/09/2026	**Probability:** 65%	**Impact:** Critical
Description: Concurrent reservation requests cause identical parking slots to be locked and confirmed for multiple motorists simultaneously.			
Refinement/Context: **Sub condition 1:** Non-atomic database writes during checkout allow overlapping allocations. **Sub condition 2:** Expired temporary locks fail to release promptly, distorting live availability grids.			
Mitigation/Monitoring: 1. Prevent double-booking by locking slots atomically in Redis during the checkout window. 2. Enforce atomic isolation on transactional database records. 3. Implement automated rollback triggers on failed checkouts.			
Management: RE computed to be Rs. 52,000 per month.			
Current Status: Mitigation steps to be initiated.			
Originator: Aryan Punalekar		**Assigned:** Yajat Padmavat	

Risk Exposure:

Risk: QR Code Spoofing & Revenue Leakage
Contactless gate access operates via optical scanning of digital QR passes generated within the mobile app. Fraudulent motorists could share static QR screenshots or reuse passes to gain unbilled access or evade overstay penalty fees.

Risk Probability: 55%

Risk Impact:
The risk impact is critical (Scale 2) because unauthorized usage directly degrades lot revenue, produces billing discrepancies, and compromises facility security.

The risk can be mitigated by encoding short-lived cryptographic timestamps (TOTP) into QR passes, ensuring tokens expire within 30 seconds and reject static screenshots, paired with optical gate induction loop sensors to permit only one vehicle cycle per validated scan. Anti-fraud verification infrastructure and gateway security cost Rs. 90,000/month.

Risk Exposure: RE $= 0.55 \times$ Rs. 90,000 $=$ Rs. 49,500/month

Risk Information Sheet

Risk ID: 3	**Date:** 12/09/2026	**Probability:** 55%	**Impact:** Critical
Description: Replay of static QR screenshots or pass duplication results in unauthorized lot entry, unbilled stays, and revenue leakage.			
Refinement/Context: **Sub condition 1:** Static barcode screenshots are shared across multiple vehicle drivers. **Sub condition 2:** Lack of synchronized check-out records allows vehicles to evade escalated hourly penalty rates.			
Mitigation/Monitoring: 1. Issue dynamically refreshing QR codes embedded with cryptographic timestamps (REQ-1). 2. Integrate optical scanners directly with barrier ground induction loops to prevent pass tailgating. 3. Log entry/exit timestamps cryptographically in PostgreSQL.			
Management: RE computed to be Rs. 49,500 per month.			
Current Status: Mitigation steps to be initiated.			
Originator: Yajat Padmavat		**Assigned:** Aditya Patil	

Risk Exposure:

Risk: Schedule Slippage Beyond 3-Month Constraint
Project design and implementation constraints mandate delivery within 3 months (CO-1). Hardware-software integration across microcontrollers, ultrasonic sensors, and full-duplex WebSockets introduces unforeseen protocol bugs that threaten the delivery timeline.

Risk Probability: 75%

Risk Impact:
The risk impact is marginal (Scale 3) relative to total facility failure, but leads to team burnout, budget overruns, and delayed commissioning.

The risk can be mitigated by constructing simulated virtual hardware APIs (MQTT emulators) allowing frontend and backend developers to build core logic in parallel without awaiting physical wiring. Automated CI/CD pipelines enforcing 80% test coverage minimize integration friction. The team sprint augmentation and testing tools cost Rs. 55,000/month.

Risk Exposure: RE $= 0.75 \times$ Rs. 55,000 $=$ Rs. 41,250/month

Risk Information Sheet

Risk ID: 4	**Date:** 12/09/2026	**Probability:** 75%	**Impact:** Marginal
Description: Friction during IoT sensor hardware and cloud backend integration causes delays exceeding the 3-month schedule cap.			
Refinement/Context: **Sub condition 1:** Hardware-level MQTT messaging issues consume developer bandwidth, stalling core app features. **Sub condition 2:** Delays in end-to-end testing reduce unit and integration coverage below the required 80% baseline.			
Mitigation/Monitoring: 1. Build virtual hardware emulators to decouple software engineering from on-site sensor delivery. 2. Execute bi-weekly Agile sprints with strict burndown tracking. 3. Enforce 80% code coverage in CI/CD pipelines before merging modules.			
Management: RE computed to be Rs. 41,250 per month.			
Current Status: Mitigation steps to be initiated.			
Originator: Aditya Patil		**Assigned:** Yajat Padmavat	

Annual Risk Management & Mitigation Budget

Risk ID	Strategy / Budget Category	Monthly Cost (Rs.)	Annual Cost (Rs.)
R1	Fail-safe mechanical relays, emergency UPS, manual override staffing	Rs. 18,000	Rs. 2,16,000
R2	High-availability Redis caching tier & distributed lock management	Rs. 14,000	Rs. 1,68,000
R3	Cryptographic dynamic QR licensing & anti-fraud security monitoring	Rs. 12,000	Rs. 1,44,000
R4	CI/CD automation, testing frameworks, and virtual hardware test harnesses	Rs. 11,000	Rs. 1,32,000
Total	Risk Management & Mitigation Reserve	Rs. 55,000	Rs. 6,60,000

Annual Operational Maintenance Budget

Category	Component Description	Monthly Cost (Rs.)	Annual Cost (Rs.)
Cloud Hosting & Infrastructure	Managed AWS EC2 instances, PostgreSQL database, and S3 receipt storage	Rs. 25,000	Rs. 3,00,000
IoT Sensor & Barrier Upkeep	Ultrasonic sensor calibration, lens cleaning, and gate barrier motor servicing	Rs. 18,000	Rs. 2,16,000
Application & API Maintenance	Bug fixes, Node.js security patches, dependency upgrades, and API maintenance	Rs. 30,000	Rs. 3,60,000
Network & Telemetry Connectivity	Broadband connections and 4G IoT SIM cards for entrance/exit barrier controllers	Rs. 7,000	Rs. 84,000
Facility Manager Support Desk	Attendant technical helpdesk and manual billing override auditing	Rs. 22,000	Rs. 2,64,000
Total	Operational Maintenance Budget	Rs. 1,02,000	Rs. 12,24,000

Combined Budget Summary

Category	Monthly Cost (Rs.)	Annual Cost (Rs.)
Risk Management & Mitigation Reserve	Rs. 55,000	Rs. 6,60,000
Operational & Hardware Maintenance	Rs. 1,02,000	Rs. 12,24,000
Total Financial Allocation	**Rs. 1,57,000**	**Rs. 18,84,000**

SE EXPERIMENT 10

Aim: To perform software testing (white-box testing) on a core module of the Smart Parking Management System (SPMS).

Theory:

White box testing (also referred to as glass box testing) evaluates the internal coding and infrastructure of software components by analysing predefined inputs against expected outputs. It focuses on inner execution pathways, conditional loops, boundary conditions, and code paths to expose hidden security flaws or broken logical structures.

White Box Testing Evaluates:

- Internal structural bugs and security vulnerabilities.
- Broken, poorly structured, or dead branches in coding logic.
- Flow of variable states and parameter inputs through the execution graph.
- Functionality and termination conditions of iterative loops.
- Statement, branch, and condition coverage on a unit/functional level.

Flow Graph Notation:

A flow graph is a directed graph $G = (V, E)$ where nodes (N) represent procedural statements or sequential basic blocks, and directed edges (E) model control transfers between statements. A predicate node (P) is a decision branch containing a condition that splits the control flow into two or more distinct paths. Enclosed bounded areas and open exterior spaces constitute non-overlapping regions (R).

Cyclomatic Complexity:

Cyclomatic complexity, $V(G)$, quantifies the logical complexity of source code and defines the exact upper bound of linearly independent execution paths required to achieve complete branch coverage. It is calculated using three standard methods:

- Method 1: $V(G) = P + 1$, where P is the count of predicate nodes.
- Method 2: $V(G) = E - N + 2$, where E is the number of directed edges and N is the total count of nodes.
- Method 3: $V(G) =$ Total number of non-overlapping regions (R) in graph G.

Advantages of White Box Testing:

- Enables precise code optimization by locating logical anomalies and dead blocks early.
- Test suites can be integrated directly into automated unit testing pipelines.
- Provides high structural confidence by verifying uncovered branches.
- Can begin implementation before external user interfaces or hardware connections are finalized.

Disadvantages of White Box Testing:

- Designing detailed white box test suites requires programming competence and code access.
- Time-intensive and complex for extensive production codebases.
- Overlooking real-world user flow variants if developer testing lacks comprehensive edge-case scenarios.

Sample Code: Gate Entry QR Verification & Barrier Control Module

The selected code implements core functional requirements from Section QR Code Entry & Exit Gate Processing and Section Fail-Safe Gate Barrier Safety SR-1 of the SPMS:

```java
public class GateController {
    public String verifyAndTriggerEntry(String qrToken, boolean isHardwareHealthy, boolean isSlotAvailable) {
        // Node 1: Entry and Initial check
        if (!isHardwareHealthy) { // Predicate Node 2
            return "FAILSAFE_MANUAL_MODE"; // Node 3
        }

        // Node 4: Decode token & check validity
        boolean isTokenValid = validateTokenSignature(qrToken);
        if (!isTokenValid) { // Predicate Node 5
            return "ACCESS_DENIED_INVALID_QR"; // Node 6
        }

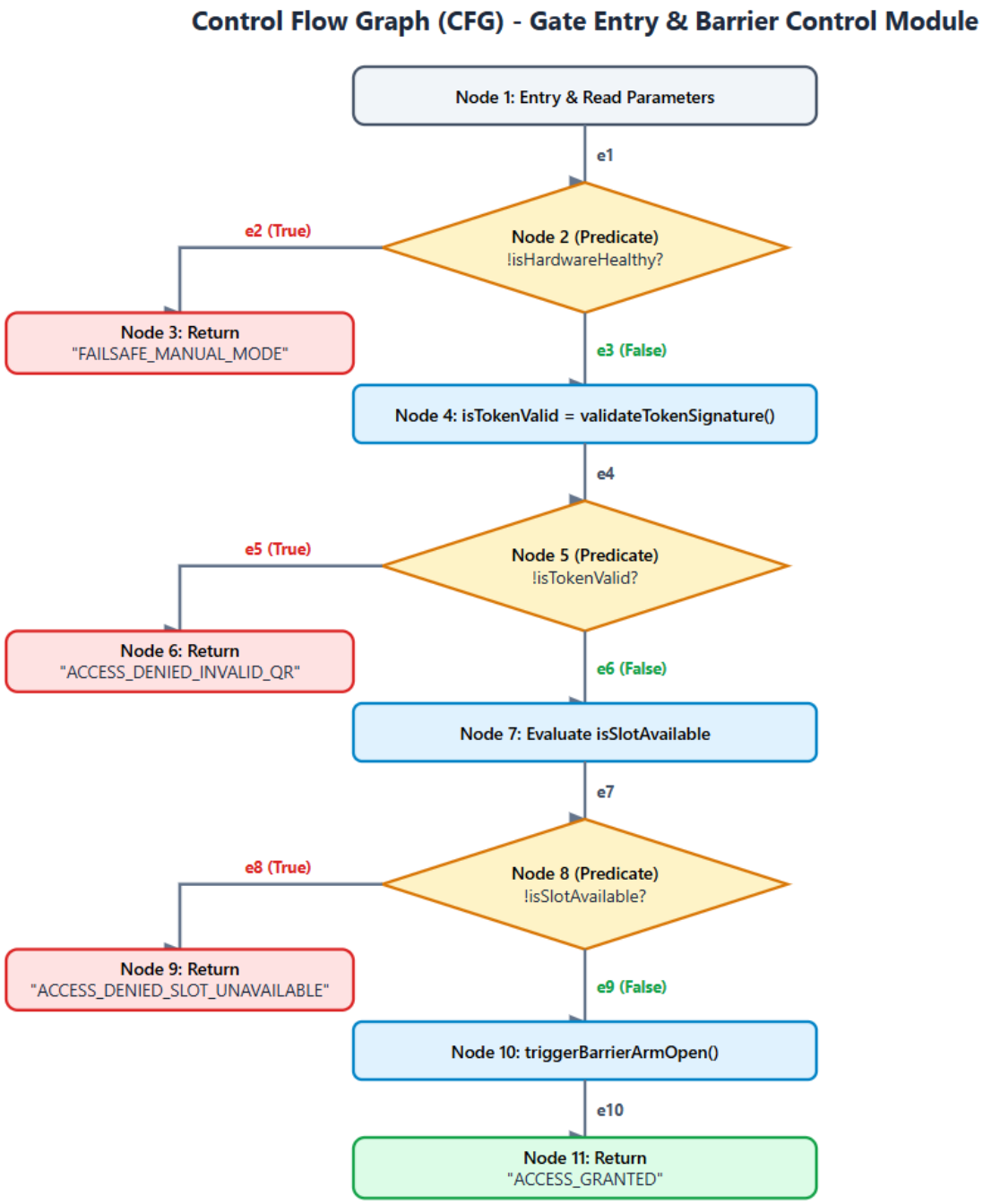
        // Node 7: Evaluate slot readiness
        if (!isSlotAvailable) { // Predicate Node 8
            return "ACCESS_DENIED_SLOT_UNAVAILABLE"; // Node 9
        }

        // Node 10: Successful validation and gate arm activation
        triggerBarrierArmOpen(); // Node 10
        return "ACCESS_GRANTED"; // Node 11
    }

    private boolean validateTokenSignature(String token) {
        return token != null && token.startsWith("SPMS-VALID");
    }

    private void triggerBarrierArmOpen() {
        // Sends MQTT open signal to ESP32 barrier relay
    }
}
```

Control Flow Representation



Calculation of Cyclomatic Complexity, $V(G)$

- Method 1 (Predicate Nodes):
  - Number of predicate nodes (P) = 3 (Nodes 2, 5, 8)
  - $V(G) = P + 1 = 3 + 1 = 4$
- Method 2 (Edges and Nodes Formula):
  - Number of directed edges (E) = 10
  - Number of nodes (N) = 8 (consolidating distinct basic-block operational and terminal nodes: 1, 2, 3, 4-5, 6, 7-8, 9, 10-11)
  - $V(G) = E - N + 2 = 10 - 8 + 2 = 4$

- Method 3 (Non-overlapping Regions):
  - Bounded closed regions (R_1, R_2, R_3) + 1 unbounded external region (R_4)
  - $V(G) = R = 4$

Independent Path Test Cases Design

Parameter / Field	Test Case 1	Test Case 2	Test Case 3	Test Case 4
Test Case ID	TC-WB-01	TC-WB-02	TC-WB-03	TC-WB-04
Target Independent Path	$1 \rightarrow 2 \rightarrow 3$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9$	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 11$
Input: QR_Token	"SPMS-VALID-99"	"INVALID-EXPIRED-TOKEN"	"SPMS-VALID-102"	"SPMS-VALID-205"
Input: isHardwareHealthy	false	true	true	true
Input: isSlotAvailable	true	true	false	true
Expected Output	"FAILSAFE_MANUAL_MODE"	"ACCESS_DENIED_INVALID_QR"	"ACCESS_DENIED_SLOT_UNAVAILABLE"	"ACCESS_GRANTED"
Execution Status	Pass	Pass	Pass	Pass

Conclusion:

White-box software testing was studied and successfully executed on the Smart Parking Management System entry and barrier control module. The control flow graph was constructed, and the Cyclomatic Complexity was evaluated to be 4 across all three analytical formulations. Four linearly independent paths were isolated, and test cases were defined to achieve 100% statement and branch test coverage.